



COURSE MATERIAL

Getting Your Machine Ready

CMSC 131 Bootcamp Block 1

Prepared by: Rene Andre Bedonia Jocsing

Published: August 26, 2026

Deadline: August 26, 2026

Prerequisites

You need this [archive](#). Unzip it somewhere permanent. Inside are three folders. `pc_asm` holds Paul Carter's source files, which saves you fetching them from a website partway through the session. `windows` and `linux` hold one checker each. You want whichever matches your machine.

What the archive doesn't contain is a finished project. You'll write the Makefile yourself and edit two of Carter's files by hand. Doing that once is how the rest of the semester stops being black magic.

Session Objectives

- Work out which of the three setup routes your laptop needs, and why macOS gets its own
- Install NASM, a 32-bit C compiler, and a bash shell, and prove each one works
- Understand what each of the three tools contributes to a single build
- Assemble, compile, link, and run your first assembly program
- Drive the whole build with one `make` command instead of four
- Set up Git and GitHub now, so the laboratory activities don't stall on it later
- Finish with `make check` reporting OK

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, `b1` through `b7`, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

Nothing. This is the first block, which assumes an ordinary laptop with an internet connection and no development tools on it at all.

Bring your own machine if you have one. Nothing below needs an administrator on a laptop set up the ordinary way. MSYS2 installs into `C:\msys64`, outside your user folder, which sounds like something Windows would stop you doing. It doesn't. Any signed-in user is allowed to create a new folder at the top of `C:\`. Creating that one folder is all the installer needs. Where this does bite is a laboratory or workplace machine that somebody has locked down, since taking that permission away is a common thing to do. Find that out at the start of the session rather than at minute seventy. If you don't have a laptop at all, say so early. Sharing one for ninety minutes works, while discovering late that you have nowhere to put your work doesn't.

The block runs ninety minutes and divides roughly like this. Installing takes about forty of them. Most

of that's waiting on downloads. Getting the project to build takes about twenty. Git and GitHub take about twenty. The last ten are for `make check` and for whatever went wrong.

The checker tells you which tools are installed and which aren't. The two copies do the same job. Which one you can run depends on how far through the block you've got.

Script	Where you run it
windows/b1_doctor.ps1	Windows PowerShell, which you already have
linux/b1_doctor.sh	Any bash, so Git Bash on Windows or your terminal on Linux

On Windows, use the PowerShell one until Part 5 finishes. There's no bash on a fresh Windows machine, so `bash b1_doctor.sh` before then answers `bash: command not found`, which looks like the checker is broken when it means the opposite. Right-click `b1_doctor.ps1` and choose **Run with PowerShell**, or run it from a PowerShell window:

```
1 powershell -ExecutionPolicy Bypass -File b1_doctor.ps1
```

On Linux, and on Windows once you have Git Bash:

```
1 bash b1_doctor.sh
```

Either one changes nothing on your machine. They look, and they explain each failure rather than only reporting it. Work down the output in order because one missing tool usually explains several complaints at once.

Part 1: What You're Actually Installing

CMSC 131 is often the first time a computer science student meets the command line, a build toolchain, and low-level programming all in the same week. That's three unfamiliar things at once, so this block separates them.

You're installing three tools that do different jobs. Knowing which is which saves you an hour later, when something fails and you need to guess what to blame.

NASM, the Netwide Assembler, turns your `.asm` file into an **object file**: machine code plus a table of names it still needs filled in. It doesn't produce a program you can run.

GCC, a C compiler, does two jobs here. It compiles `driver.c`, a tiny C program that calls into your assembly, and it acts as the **linker**, stitching object files together into a runnable program.

A bash shell is where you type. On Linux you already have one. On Windows you'll install Git Bash. Windows ships PowerShell, which mangles some of the arguments the linker needs. This course avoids that fight entirely by using bash instead.

Why does an assembly course need a C compiler? Writing a program that the operating system can start on its own means handling process startup, the C runtime, and standard output yourself. That's a lot of ceremony before you can print a number.

Instead, [Paul Carter's library](#) gives you `driver.c`, which is five lines long:

```
1 #include "cdecl.h"
2
3 int PRE_CDECL asm_main( void ) POST_CDECL;
4
```

```
5  int main()
6  {
7      int ret_status;
8      ret_status = asm_main();
9      return ret_status;
10 }
```

C starts the process and immediately calls `asm_main`. That's your assembly. Everything after that is yours. When you return, C shuts the process down cleanly. You get to write the interesting part and skip the ceremony.

Your program will be **32-bit**. That's why every command below has `-m32`, `win32`, or `elf32` in it. Mixing a 32-bit object file with a 64-bit one produces a linker error that reads like nonsense, so when you see one, check this first.

Part 2: Which Machine You're Doing This On

Before installing anything, work out which of three routes you're on. They diverge more than you'd expect. Picking the wrong one costs you an afternoon.

Windows is the common case. It's also what the rest of this manual assumes when it doesn't say otherwise. Everything installs natively. Go to Part 3.

Linux is the shortest route of the three. Your shell already exists, your package manager has every tool, and the whole install is two commands. Go to Part 3.

macOS can't run this course's programs at all. That isn't a gap in these instructions. Two separate things are in the way. macOS stopped being able to execute 32-bit programs in Catalina, released in 2019. And if your Mac has an Apple Silicon chip, an M1 or later, it can't run x86 instructions at all, which is what this entire course is about. Rosetta doesn't rescue you here either. It translates 64-bit x86. Your programs are 32-bit.

So on macOS you run **Linux inside a virtual machine**, which is a whole simulated computer running as a program on your Mac, and then follow the Linux instructions from inside it. Read Part 2.1 and then go to Part 3.

Part 2.1: The Virtual Machine, for macOS Only

Install [UTM](#), which is free. When you create the virtual machine, you must choose an **x86_64** Debian or Ubuntu image. An ARM64 or `aarch64` image won't work.

That's the one decision in this block that's easy to get wrong and painful to discover late, so read the two options before you click. UTM offers you two modes:

- **Virtualize** runs a guest built for your Mac's own chip. On Apple Silicon that means an ARM64 Linux. It boots in seconds and feels great. Then `nasm` produces x86 machine code that the ARM processor underneath can't execute, and every program you write this semester fails to run.
- **Emulate** pretends to be a different processor. This is the one you want. Choose Emulate, and choose an `x86_64` image.

On an Intel Mac, Virtualize already gives you `x86_64`, so use it and enjoy the speed.

Emulation is slower. You'll feel it during installation and boot, but barely afterwards. The programs in this course are a few hundred lines that assemble in well under a second, so the penalty lands on a one-time setup rather than on your daily work.

Once Linux is running inside UTM, everything below marked **Linux** is what you follow. Type it in the virtual machine's terminal, not in the macOS Terminal.

Part 3: NASM

Windows

1. Go to the [NASM](#) website and follow **Download**, which puts you in the release builds directory. Nothing there is named `stable`, so take the highest plain version number. Ignore anything with `rc` in it, which marks a release candidate rather than a finished release. The front page links that same version directly. At the time of writing it's 3.02.
2. Open its `win64` folder and download the file whose name ends `installer-x64.exe`. The `.zip` sitting beside it holds the same assembler without an installer, so leave it alone.
3. Run it, and **note where it installs**. The default is `C:\Users\<you>\AppData\Local\bin\NASM`.
4. Add that folder to your `PATH`, the list of folders your system searches when you type a command. Press the Windows key, type `environment variables`, open **Edit the system environment variables**, click **Environment Variables**, select `Path` under *User variables*, click **Edit**, then **New**, and paste the folder from step 3.

Linux (Debian/Ubuntu)

```
1 sudo apt update
2 sudo apt install -y nasm
```

Verify. Open a fresh terminal and run:

```
1 nasm -v
```

You should see a version line such as `NASM version 3.02`. If it says “`nasm` is not recognized” right after installing on Windows, then that's because `PATH` is read when a terminal starts. A terminal you opened before the install still has the old one. Close it, open a new one, try again. It's the most common failure in this block. The same fix applies to every tool below.

Part 4: The 32-bit Toolchain

`GCC`, the debugger, and `make` all arrive together.

Windows

They come from `MSYS2`, a package manager for Windows. Fetching it, and fetching `Git` in Part 5, both go through **WinGet**, which is Windows' own installer for command-line programs. Find out now whether you have it. That determines which of the two routes below you take:

```
1 winget --version
```

A version number means you're set. Go to step 1.

“Not recognized” doesn't mean anything is broken. `WinGet` arrives inside a Windows component called **App Installer**, which a few ordinary things can delay. On a machine that has only just been signed into for the first time, the Microsoft Store registers App Installer in the background, so `WinGet` doesn't exist until that finishes. Ask for the registration directly rather than waiting:

```
1 Add-AppxPackage -RegisterByFamilyName -MainPackage
   ↪ Microsoft.DesktopAppInstaller_8wekyb3d8bbwe
```

If App Installer is missing outright, get it from the [Microsoft Store](#), open a new PowerShell window, and ask for the version again. One kind of machine can't be rescued that way. `WinGet` needs Windows 10 version 1809 or newer, which you can read under **Settings**, then **System**, then **About**. On anything older it doesn't run.

Nor does installing help on a laboratory or work machine where somebody has removed the Store, or set a policy against WinGet. Both look the same from PowerShell as a machine that never had it.

None of that stops you. Both things this manual installs with WinGet also ship as ordinary installers you download and double-click:

- MSYS2, from [msys2.org](https://www.msys2.org). Take the file named `msys2-x86_64-` followed by a date, run it, accept the folder it offers, and carry on at step 2 below. Step 4 assumes that default folder. MSYS2 wants 64-bit Windows 10 version 1809 or newer, the same floor WinGet has.
- Git for Windows, from gitforwindows.org, which is what Part 5 asks for.

Nothing after the install differs between the two routes.

1. Open **PowerShell** and install MSYS2:

```
1 winget install --id MSYS2.MSYS2 --exact
```

2. Close PowerShell. Open **MSYS2 UCRT64** from the Start menu and update it:

```
1 pacman -Syu
```

If it tells you to close the terminal, do that, reopen it, and run `pacman -Syu` once more.

3. Install the 32-bit toolchain group:

```
1 pacman -S --needed mingw-w64-i686-toolchain
```

Accept the default when it offers to install every member of the group. That group is where your compiler, your debugger, and your `make` all come from.

4. Add `C:\msys64\mingw32\bin` to your PATH, the same way you added NASM in Part 3. Read down the list before you add anything. A machine that has had MSYS2 on it before may still carry the entry. A second copy of it does nothing.

`mingw-w64-i686-toolchain` is a package group. Installing it gets you `mingw-w64-i686-gcc`, `mingw-w64-i686-gdb`, and `mingw-w64-i686-make` in one command. Block 4 uses the debugger and every block from here uses `make`, so this one line covers the rest of the semester.

Step 4 has a trap in it for anyone whose laptop already carried a compiler, from Strawberry Perl or Anaconda or an Android toolchain. PATH isn't one list. Windows builds the one a program sees by pasting the system list and your user list together, in that order, so every folder under *System variables* gets searched before the first folder under *User variables*. Part 3 had you add NASM under *User variables*. Put MSYS2 there too and it sits behind any 64-bit `gcc` that was already in the system list. `gcc -dumpmachine` goes on answering `x86_64-w64-mingw32`, so it looks like the step didn't take. It did. The other compiler is ahead of it. Nothing you do under *User variables* reaches past that. What fixes it is moving `C:\msys64\mingw32\bin` to the top of the system list. That edit is the one thing in this block that needs an administrator.

Linux (Debian/Ubuntu)

```
1 sudo apt install -y gcc-multilib gdb make
```

The interesting package there is `gcc-multilib`. Your Linux is almost certainly 64-bit. A 64-bit compiler by default has only 64-bit versions of the system libraries to link against. Multilib adds the 32-bit ones alongside them, which is what `-m32` needs. Without it, `gcc -m32` fails with a complaint about a

missing header or a missing library rather than anything that mentions 32 bits, so if you see that, this is the package you skipped.

Verify.

On **Windows**, in a fresh Git Bash window once Part 5 is done:

```
1 gcc -dumpmachine
2 mingw32-make --version
3 gdb --version
```

The first must print i686-w64-mingw32. If it prints something with x86_64 in it, you have a 64-bit compiler ahead of the 32-bit one on PATH. Nothing in this course will link until you fix that ordering. To see who's winning, ask for all of them:

```
1 type -a gcc
```

That lists every gcc on your PATH in the order bash would try them, so the first line is the one you've been getting. If it refers to a folder you didn't put there, read the paragraph in step 4 about the two lists. It explains why your first attempt at fixing this may have changed nothing.

On **Linux**, gcc -dumpmachine printing x86_64-linux-gnu is correct. One compiler targets both widths, so there's no separate 32-bit one to install. Prove the 32-bit half directly:

```
1 echo 'int main(void){return 0;}' > /tmp/m32check.c
2 gcc -m32 /tmp/m32check.c -o /tmp/m32check && echo "32-bit builds work"
3 make --version
4 gdb --version
```

If that first command errors, you're missing gcc-multilib.

Part 5: The Shell, and One Command Name

Windows

1. Install Git for Windows from PowerShell. It gives you both the shell and the version control you'll set up in Part 6:

```
1 winget install --id Git.Git --exact --source winget
```

If Part 4 told you this machine has no WinGet, download the installer from gitforwindows.org instead and take its defaults. The rest of this part reads the same either way.

2. Open **Git Bash** from the Start menu. This is where you run every command in this course from now on.
3. The make that MSYS2 installed is named mingw32-make, not make. Every other machine in the world calls it make, and so does the rest of this course, so give yours the same name:

```
1 echo "alias make='mingw32-make'" >> ~/.bashrc
```

Close Git Bash and open it again, so the change takes effect.

This step isn't optional. From Block 2 onward the manuals say `make`, and they mean the command you just aliased.

Two things about that alias will trip you up. It only exists in an interactive shell. A script run with `bash something.sh` can't see it. The checker in this block's archive therefore looks for `mingw32-make` and asks you to test `make --version` yourself. And if `make --version` already worked before you ran that line, you have another `make` on your machine from something unrelated. Tools like `devkitPro` bring one along. That isn't a problem here. The Makefile in Part 7 asks `uname` as well as checking the OS, which is the case it was written for. But it's the first thing to suspect if a build misbehaves in a way nothing else explains.

Verify.

```
1 bash --version
2 diff --version
3 make --version
```

`diff` matters more than it looks. It's what `make check` uses to compare your program's output against the expected output. It comes from Git Bash rather than from MSYS2.

A common gotcha is assuming that Git Bash includes a compiler. It doesn't. Git for Windows ships a shell and Unix utilities, no `gcc` or `make`. If you skipped Part 4 because you already had Git Bash, go back.

Linux

Your terminal is already `bash` and your `make` is already called `make`, so there's nothing to alias. Install Git if it isn't there:

```
1 sudo apt install -y git
```

Verify.

```
1 bash --version
2 diff --version
3 make --version
4 git --version
```

Part 6: Git and GitHub

Nothing in this block needs version control. The laboratory activities need all of it. Each group keeps one public GitHub repository. Your individual score is computed from the commits you personally make to it. Groups form the moment the bootcamp ends, which is roughly a month from now.

So do this today, in the laboratory room, where someone can look at your screen when it misbehaves. Week five is a bad time to discover you can't push.

Step 1: An Account

If you already have a GitHub account, skip to step 2.

Otherwise, sign up at github.com and verify your email address. Choose the username seriously. It appears on every commit you make, your instructor reads it while grading, and it tends to outlive the course by about a decade. Something recognisably you is a better long-term choice than a joke you'll want to change after you graduate.

While you're there, look at the [GitHub Student Developer Pack](#). It's free for enrolled students and it's worth spending ten minutes on.

Step 2: Two-Factor Authentication

GitHub requires two-factor authentication for accounts that contribute code, so this isn't a step you can skip. Turn it on now, from **Settings**, then **Password and authentication**.

Save your recovery codes somewhere that isn't the phone running your authenticator app. Losing both at once is a miserable way to lose access to a semester of work.

Step 3: Tell Git Who You Are

```
1 git config --global user.name "Your Full Name"
2 git config --global user.email "your.email@example.com"
```

Use the email attached to your GitHub account. Commits carry whatever address is configured here. GitHub matches that address against accounts to decide who wrote what. Get it wrong and your commits show up as belonging to nobody.

That matters for your grade. The syllabus has your repository audited for individual commit contributions at the end of each activity, so a commit GitHub can't attribute to you is, for grading purposes, a commit you didn't make. Read the syllabus section on the Individual Contribution Score for how it's computed. Don't commit from a shared account.

Step 4: Let Your Machine Push

This is the step that catches people out (it certainly caught me out). GitHub stopped accepting passwords for Git over HTTPS in 2021. Your first `git push` will ask for a password, refuse the one you type, and tell you techno-gibberish that doesn't obviously mean "passwords don't work anymore."

Windows

Git Credential Manager came with Git for Windows in Part 5, so this is nearly automatic. The first time you push, a browser window opens, you authorize it once, and your credentials are stored from then on. There's nothing to install.

Linux, including the macOS virtual machine

No credential helper is bundled, so use an SSH key. That's a pair of files: a private one that stays on your machine and a public one you hand to GitHub.

```
1 ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press Enter three times to accept the default location and an empty passphrase. Then print the public half:

```
1 cat ~/.ssh/id_ed25519.pub
```

GitHub's own instructions suggest choosing a passphrase there. This manual doesn't, so here's the trade you're making. A passphrase makes the key useless to anyone who walks off with your laptop. It also means typing that passphrase on every push, unless you set up an agent to hold it for you. On a machine only you use, most people take the empty one. Nothing about that is permanent. `ssh-keygen -p -f ~/.ssh/id_ed25519` adds or changes a passphrase later without generating a new key, and GitHub never sees that half of the pair anyway.

Copy that entire line, then on GitHub go to **Settings**, then **SSH and GPG keys**, then **New SSH key**, and paste it. The form asks what kind of key this is. Choose the authentication kind rather than the signing kind. Signing keys prove a commit came from you, which is a separate feature this course doesn't use. Confirm it worked:


```
1  ssh -T git@github.com
```

GitHub answers with your username and a note that it doesn't provide shell access. That message is success, despite sounding like a rejection.

Be careful about which of the two files you paste. The one ending in `.pub` is the public half and is meant to be shared. The one without an extension is private and should never leave your machine.

Verify, on either platform. Make a throwaway repository on GitHub, clone it, commit something, and push:

```
1  git clone <your repository URL>
2  cd <repository name>
3  echo "hello" > test.txt
4  git add test.txt
5  git commit -m "test: check that pushing works"
6  git push
```

If that push succeeds, you're ready for the laboratory activities. Delete the repository afterwards.

Part 7: The Project

1. Go to the [pcasm](#) site, find **Example Code**, and download the **MS C Examples** archive. Despite the name, the source files in it are the ones both platforms use.
2. Extract it into a folder named `pc_asm`. Put that folder somewhere you'll still have it in December, such as `~/cmssc131/pc_asm`, and not in Downloads. Every remaining block builds inside it. A folder you clear out over the semester break takes the whole bootcamp with it.
3. Create a file in that folder called `Makefile`, with no extension, containing this:

```
1  # Which platform is this? Ask twice, for the reason given below.
2  UNAME := $(shell uname -s 2>/dev/null)
3
4  ifeq ($(OS),Windows_NT)
5      PLATFORM := windows
6  else ifeq ($(findstring MINGW,$(UNAME)))
7      PLATFORM := windows
8  else ifeq ($(findstring MSYS,$(UNAME)))
9      PLATFORM := windows
10 else ifeq ($(findstring CYGWIN,$(UNAME)))
11     PLATFORM := windows
12 else
13     PLATFORM := $(UNAME)
14 endif
15
16 NASM := nasm
17 CC   := gcc
18
19 ifeq ($(PLATFORM),windows)
20     ASFLAGS := -f win32
21     LDFLAGS := -Wl,-subsystem,console
22     EXE     := .exe
23 else
24     ASFLAGS := -f elf32 -d ELF_TYPE
25     LDFLAGS := -no-pie
26     EXE     :=
27 endif
```

```

28
29 CFLAGS := -m32
30
31 PROG ?= skel
32 BIN := $(PROG)$(EXE)
33
34 $(BIN): $(PROG).obj asm_io.obj driver.o
35     $(CC) $(CFLAGS) $^ -o $@ $(LDFLAGS)
36
37 %.obj: %.asm
38     $(NASM) $(ASFLAGS) $< -o $@
39
40 driver.o: driver.c cdecl.h
41     $(CC) $(CFLAGS) -c $< -o $@
42
43 run: $(BIN)
44     ./$(BIN)
45
46 STDIN := $(if $(wildcard $(PROG).input),< $(PROG).input,)
47
48 replay: $(BIN)
49     @./$(BIN) $(STDIN)
50
51 check: $(BIN)
52     @./$(BIN) $(STDIN) | diff -u --strip-trailing-cr --label "$$(PROG).expected" --label
    ↪ "what $(PROG) printed" $(PROG).expected - && echo "OK: $(PROG) matches
    ↪ $(PROG).expected"
53
54 clean:
55     rm -f *.obj *.o *.exe $(PROG)
56
57 .PHONY: run replay check clean

```

Those indented lines must be **tabs**. Make is strict about this. The error it gives you when they're spaces says `missing separator`.

Read that one line about `STDIN` twice. It's the difference between a check that works and a check that hangs. `skel` reads nothing, so it does nothing today. From Block 2 on, your programs ask the user for numbers. A program waiting on a keyboard that nobody is typing at will sit there forever while `make` waits on it. When a file called `convert.input` exists next to `convert.asm`, that line quietly feeds it to the program instead.

`replay` is the same `run` check performs with the comparison left off, so it prints what `check` reads and leaves the judging to you. Today that buys you nothing, since `skel` reads nothing at all and `replay` does the same as `run`. Block 2 is where it starts earning its place. It explains itself there.

Why does the top of that file ask which platform you're on twice? Because neither question answers it alone. Windows sets an environment variable called `OS`. That's the quick check, but a `make` built for `MSYS2` or `Cygwin` reports that variable as empty even while running on Windows. You can end up with one of those by accident. So the file falls back to asking `uname`, which those builds answer with a name starting `MINGW`, `MSYS`, or `CYGWIN`. Without the second question, such a `make` picks the Linux branch on a Windows machine and the build fails several steps later, complaining about something unrelated. This isn't hypothetical. It happened while testing this manual.

4. Open `asm_io.inc` and add these four lines at the very top, above everything else:

```

1  #ifndef ELF_TYPE
2      #define _asm_main asm_main
3      #section ".note.GNU-stack" noalloc noexec nowrite progbits
4  #endif

```

Then open `asm_io.asm` and add the same block near the top, just after the `%define OF_MASK` line. That file doesn't include `asm_io.inc`, so it needs its own copy.

5. Build and run:

```
1 make run
```

You should see `Hello, world!`.

Why Step 4 Exists

Carter wrote his library before you had to care. The two operating systems disagree about names. When C on Windows exports a function called `asm_main`, the name that lands in the object file is `_asm_main`, with a leading underscore. Linux uses the name as written, with no underscore.

Left alone, that would mean keeping two versions of every program you write this semester, differing by one character. Those four lines mean you don't have to. You write global `_asm_main` and `_asm_main`: on either platform. On Linux, the assembler quietly rewrites it.

The `.note.GNU-stack` line does a separate job. Newer Linux linkers warn about any object file that doesn't say whether it wants an executable stack. Yours doesn't want one. The warning is harmless, but it appears on every build until you say so. A warning you've learned to ignore is a warning you'll still be ignoring when it matters. Both files need the line because the linker inspects each object separately.

What the Makefile Ran

The Makefile just ran four commands on your behalf. On Windows:

```
1 nasm -f win32 skel.asm -o skel.obj
2 nasm -f win32 asm_io.asm -o asm_io.obj
3 gcc -m32 -c driver.c -o driver.o
4 gcc -m32 skel.obj asm_io.obj driver.o -o skel.exe -Wl,-subsystem,console
```

On Linux:

```
1 nasm -f elf32 -d ELF_TYPE skel.asm -o skel.obj
2 nasm -f elf32 -d ELF_TYPE asm_io.asm -o asm_io.obj
3 gcc -m32 -c driver.c -o driver.o
4 gcc -m32 skel.obj asm_io.obj driver.o -o skel -no-pie
```

The first two assemble your program and Carter's I/O library into object files. The third compiles the C driver. The fourth links all three into a program you can run.

Three differences explain the Makefile's two branches. Windows object files use a format called COFF and Linux uses one called ELF, which is `-f win32` against `-f elf32`. Windows needs `-Wl,-subsystem,console` to say this is a command-line program and not a windowed one. And Linux needs `-no-pie`. GCC builds programs that can be loaded at any address by default, while the addressing this course teaches assumes fixed ones. Leave `-no-pie` out on Linux and the link fails with a message about relocation `R_386_32`, which tells you almost nothing unless you already knew this paragraph.

Working on Something Other Than `skel`

`PROG` selects which program to build. It defaults to `skel`, so you override it:

```
1 make PROG=lab1 run
```

Something to Write Code In

Nothing so far has said what to open these files in. Until now you've been typing commands. From here on you're writing assembly, so you need an editor that doesn't fight you.

Visual Studio Code is the safe default. It's free, it runs on all three platforms, and it has an extension called **The Netwide Assembler (NASM)** that colours `.asm` files so a mistyped instruction looks wrong before you assemble it. Install the editor, then search the Extensions panel for that name.

Whatever you use, it has to be able to type a tab character rather than spaces. Most editors helpfully convert one into the other. A Makefile with spaces where it wants tabs fails with `missing separator`, which is the same error you'd get from a much worse mistake. In VS Code, open the Makefile and click the **Spaces: 4** indicator in the status bar along the bottom, choose **Indent Using Tabs**, and the file will behave.

Despite the memes, a word processor isn't an editor. Word and Google Docs insert curly quotes and invisible formatting that the assembler can't read. The errors it gives you won't mention any of that.

If Yours Won't Build

There's a finished, working version of this project at [cm-sc-131-lab0-nasm](https://github.com/WhiteLicorice/cm-sc-131-lab0-nasm). It has the Makefile above, the `asm_io.inc` change already applied, and a record of what was tested on which platform. Read it when your own setup misbehaves and you want something known to work to compare against.

Clone it next to your own folder rather than on top of it:

```
1 git clone https://github.com/WhiteLicorice/cm-sc-131-lab0-nasm.git
```

Later blocks send you back here. If you ever miss a session and need a project that works so you can get on with the day's material, that clone is it.

Part 8: Proving It Works

Running a program and eyeballing the output is a weak check. You'll change something, glance at the terminal, and see what you expect to see instead of what's there. So the finish line for this block is a comparison a machine performs.

1. Create a file called `skel.expected` holding what `skel.asm` prints, byte for byte. That's the line `Hello, world!`, then a blank line. In an editor that means typing the text, pressing Enter twice, and saving. If you don't trust your editor about invisible characters (it needs to be byte for byte!), make the file from the shell instead:

```
1 printf 'Hello, world!\n\n' > skel.expected
```

Getting this wrong is the most common way to fail a check that should pass. A missing or extra newline at the end of a file looks like nothing at all on screen.

2. Run:

```
1 make check
```

Success looks like this:

```
1 OK: skel matches skel.expected
```

You strip carriage returns during checks because on Windows your program writes Windows line endings, a carriage return followed by a newline (`\r\n`), while your `skel.expected` file almost certainly

holds Unix line endings (`\n`). A plain `diff` then reports every single line as different while displaying two lines that look identical, which is a maddening thing to debug.

That's what `--strip-trailing-cr` in the `check` recipe is for. It does nothing on Linux, where both sides already agree, so it stays in the recipe on both platforms rather than becoming one more thing that varies. Every activity in this course that compares output uses it.

Testing Checklist

Run the checker from the archive and it does the first five of these for you, with a diagnosis attached to anything that fails. The `make` half of the third line is the one gap, since a script can't see an alias. The rest you do by hand, starting with the `push`. No script can tell whether yours reached GitHub.

Core Functionality

- `nasm -v` prints a version
- `gcc -dumpmachine` prints `i686-w64-mingw32` on Windows, or `gcc -m32` builds a test program without error on Linux
- `make --version` and `gdb --version` both print versions
- `bash --version` and `diff --version` both print versions
- `git config user.name` and `git config user.email` both print what you set
- A push to a throwaway GitHub repository succeeds
- `make run` prints `Hello, world!`
- `make check` prints `OK: skel matches skel.expected`
- `make clean` removes the object files and the program
- Running `make check` again after `clean` rebuilds and still passes

If every line above holds, you're done with Block 1.

Common Pitfalls

- missing separator from `make` means your Makefile has spaces where it needs tabs
- undefined reference to `asm_main` means your `.asm` file is missing `global _asm_main`, or you assembled with the wrong `-f` format
- cannot find `-lmingw32`, arriving at the end of a wall of skipping incompatible ... when searching for lines, means your `gcc` is a 64-bit one that has no 32-bit libraries beside it. Assembling and compiling both succeeded, which is why nothing in the message mentions widths. Check `gcc -dumpmachine`
- i386 architecture of input file ... is incompatible with i386:x86-64 output is the neighbouring mistake, where `-m32` fell off the link line rather than off the whole build, so check the last recipe in your Makefile
- On Linux, relocation `R_386_32` ... can not be used when making a PIE object means `-no-pie` is missing from your link line
- On Linux, `gcc -m32` failing about a missing header or library means you skipped `gcc-multilib`
- Support for password authentication was removed means you skipped Part 6 step 4, so your machine has no way to prove who it is
- On an Apple Silicon Mac, a program that assembles but won't run means your virtual machine is ARM64 instead of `x86_64`
- `check` fails but the two outputs look the same: line endings, and `--strip-trailing-cr` is missing from your recipe
- `winget` answering "not recognized" is App Installer missing rather than anything you did. Part 4 opens with what to do about it, including the route that skips WinGet entirely
- Any "command not found" immediately after an install: stale terminal, open a new one
- `gdb` that installs but then exits complaining about a shared library it can't name means a half-finished update, where the debugger is newer than a library it depends on. Run `pacman -Syu` in MSYS2 until it says there's nothing left to do. Block 4 is built on `gdb`, so fix it now rather than in three weeks

Architecture Review

What We Built

- A three-tool toolchain where each tool has one clear job
- A four-step build reduced to one command that reads the same on every platform
- An automated correctness check rather than a visual one
- A GitHub account your commits can be attributed to
- A project layout that every later activity reuses unchanged

Key Takeaways

1. **Assembler, compiler, linker are three different things.** Knowing which one produced an error tells you where to look.
2. **The C driver exists to skip process-startup ceremony,** not because assembly needs C.
3. **32-bit is a hard requirement here.** Half of all confusing linker errors are a 64-bit tool sneaking in. On a Mac it's the reason you need a virtual machine at all.
4. **PATH is read at terminal start.** Open a new terminal before believing an install failed.
5. **The same source builds on both platforms because four lines reconcile a naming difference.** You've just stopped having to think about it.
6. **A machine-checked result beats a glance at the screen.** `check` is the habit this course builds on.

Next Session Preview

- What a register is, and why only eight registers exist
- Reading a number from the user and printing one back
- Arithmetic that only works on whole numbers, and what happens to the remainder
- Building a temperature converter from an empty file